

```

"""
Backblaze B2 MCP Server
=====

FastMCP server for the Backblaze B2 cloud storage API (native B2 + S3-compatible).

Auth env vars:
    B2_APPLICATION_KEY_ID      - Application Key ID (or Master Key ID)
    B2_APPLICATION_KEY         - Application Key (secret)

The server auto-authorizes on first use and re-authorizes on token expiry.
"""

import os
import json
import base64
import hashlib
import time
import httpx
from typing import Optional, Dict, Any, List
from pydantic import BaseModel, Field, ConfigDict
from mcp.server.fastmcp import FastMCP

# — Constants —————
KEY_ID = os.environ.get("B2_APPLICATION_KEY_ID", "")
APP_KEY = os.environ.get("B2_APPLICATION_KEY", "")
TIMEOUT = 120.0
B2_AUTH_URL = "https://api.backblazeb2.com/b2api/v3/b2_authorize_account"

mcp = FastMCP("backblaze_mcp")

# — Auth state (module-level cache) —————
_auth: Dict[str, Any] = {}
_auth_ts: float = 0.0
AUTH_TTL = 3600 * 20 # re-auth after 20 hours

async def _authorize() -> Dict[str, Any]:
    global _auth, _auth_ts
    if _auth and (time.time() - _auth_ts) < AUTH_TTL:
        return _auth
    if not KEY_ID or not APP_KEY:
        raise RuntimeError("B2_APPLICATION_KEY_ID and B2_APPLICATION_KEY must be
        creds = base64.b64encode(f'{KEY_ID}:{APP_KEY}'.encode()).decode()
        async with httpx.AsyncClient(timeout=TIMEOUT) as client:

```

```

        r = await client.get(B2_AUTH_URL,
                             headers={"Authorization": f"Basic {creds}"})
        r.raise_for_status()
        _auth = r.json()
        _auth_ts = time.time()
    return _auth

async def _b2(method: str, endpoint: str,
              body: Optional[Dict] = None,
              stream_response: bool = False) -> Any:
    auth = await _authorize()
    api_url = auth["apiInfo"]["storageApi"]["apiUrl"]
    token = auth["authorizationToken"]
    url = f"{api_url}/b2api/v3/{endpoint}"
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        fn = client.post if method == "POST" else client.get
        r = await fn(url, headers={"Authorization": token},
                     json=body if method == "POST" else None,
                     params=body if method == "GET" else None)
        if r.status_code == 401:
            # force re-auth
            global _auth_ts
            _auth_ts = 0
            return await _b2(method, endpoint, body, stream_response)
        r.raise_for_status()
    return r.json()

def _fmt_size(b: int) -> str:
    for unit in ["B", "KB", "MB", "GB", "TB"]:
        if b < 1024:
            return f"{b:.1f} {unit}"
        b /= 1024
    return f"{b:.1f} PB"

# — Input Models —————
class ListFilesInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    bucket_id: str = Field(..., description="Backblaze B2 bucket ID")
    prefix: str = Field(default="", description="Filter files by key prefix (fold
    max_count: int = Field(default=100, description="Max files to return", ge=1,
    start_file_name: Optional[str] = Field(default=None, description="Pagination

class FileInfoInput(BaseModel):

```

```

model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
file_id: str = Field(..., description="B2 file ID (starts with '4_')")

class DeleteFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="B2 file ID to delete")
    file_name: str = Field(..., description="Full file name/key (required by B2 A

class CopyFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    source_file_id: str = Field(..., description="Source file ID")
    dest_bucket_id: str = Field(..., description="Destination bucket ID")
    dest_file_name: str = Field(..., description="Destination file name/key")

class CreateBucketInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    bucket_name: str = Field(..., description="New bucket name (globally unique)")
    bucket_type: str = Field(default="allPrivate", description="'allPrivate' or '

class BucketIdInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    bucket_id: str = Field(..., description="Backblaze B2 bucket ID")

class DownloadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="B2 file ID to download")
    local_path: str = Field(..., description="Absolute local destination path")

class UploadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    local_path: str = Field(..., description="Absolute local path to file")
    bucket_id: str = Field(..., description="Destination bucket ID")
    dest_key: Optional[str] = Field(default=None, description="Destination key/pa

class HideFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    bucket_id: str = Field(..., description="Bucket ID")
    file_name: str = Field(..., description="File name to hide (creates a hide ma

```

```

# — Tools —————
@mcp.tool(name="b2_get_account_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def b2_get_account_info() -> str:
    """Return Backblaze B2 account info: account ID, API URL, and download URL."""
    try:
        auth = await _authorize()
        lines = [
            "## Backblaze B2 Account",
            f"- **Account ID**: {auth.get('accountId')}",
            f"- **API URL**: {auth['apiInfo']['storageApi']['apiUrl']}",
            f"- **Download URL**: {auth['apiInfo']['storageApi']['downloadUrl']}"
            f"- **Recommended Part Size**: {_fmt_size(auth['apiInfo']['storageApi']",
            f"- **Absolute Min Part Size**: {_fmt_size(auth['apiInfo']['storageAp
        ]
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="b2_list_buckets",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def b2_list_buckets() -> str:
    """List all Backblaze B2 buckets on the account."""
    try:
        auth = await _authorize()
        data = await _b2("POST", "b2_list_buckets", {"accountId": auth["accountI
        buckets = data.get("buckets", [])
        if not buckets:
            return "No buckets found."
        lines = [f"## B2 Buckets ({len(buckets)})", ""]
        for b in buckets:
            lines.append(f"- **{b['bucketName']}** (id=`{b['bucketId']}`, type={
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="b2_list_files",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def b2_list_files(params: ListFilesInput) -> str:
    """List files in a B2 bucket, optionally filtered by prefix."""
    try:
        body: Dict[str, Any] = {
            "bucketId": params.bucket_id,
            "maxFileCount": params.max_count,
            "prefix": params.prefix,

```

```

    }
    if params.start_file_name:
        body["startFileName"] = params.start_file_name
    data = await _b2("POST", "b2_list_file_names", body)
    files = data.get("files", [])
    nxt = data.get("nextFileName")
    if not files:
        return f"No files found in bucket `{params.bucket_id}`" + (f" with pr
lines = [f"## Files in bucket `{params.bucket_id}`, f"***{len(files)} fil
for f in files:
    lines.append(f"- `{f['fileName']}` {_fmt_size(f.get('contentLength'),
if nxt:
    lines.append(f"\n_More files after `{nxt}`. Use `start_file_name` to
return "\n".join(lines)
except Exception as e:
    return f"Error listing files: {e}"

@mcp.tool(name="b2_get_file_info",
    annotations={"readOnlyHint": True, "destructiveHint": False})
async def b2_get_file_info(params: FileInfoInput) -> str:
    """Get detailed metadata for a B2 file by its file ID."""
    try:
        data = await _b2("POST", "b2_get_file_info", {"fileId": params.file_id})
        lines = [
            "## File Info",
            f"- **Name**: {data.get('fileName')}",
            f"- **File ID**: `{data.get('fileId')}`",
            f"- **Bucket ID**: `{data.get('bucketId')}`",
            f"- **Size**: {_fmt_size(data.get('contentLength', 0))}",
            f"- **Content-Type**: {data.get('contentType')}",
            f"- **SHA1**: `{data.get('contentSha1')}`",
            f"- **Uploaded**: {data.get('uploadTimestamp')}",
            f"- **Action**: {data.get('action')}",
        ]
        if data.get("fileInfo"):
            lines.append(f"- **Custom Info**: {json.dumps(data['fileInfo'])}")
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="b2_create_bucket",
    annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def b2_create_bucket(params: CreateBucketInput) -> str:
    """Create a new Backblaze B2 bucket."""
    try:

```

```

    auth = await _authorize()
    data = await _b2("POST", "b2_create_bucket", {
        "accountId": auth["accountId"],
        "bucketName": params.bucket_name,
        "bucketType": params.bucket_type,
    })
    return f"✅ Bucket created: **{data['bucketName']}** (id=`{data['bucketId']}`)"
except Exception as e:
    return f"Error creating bucket: {e}"

@mcp.tool(name="b2_delete_bucket",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempotentHint": True},
          async def b2_delete_bucket(params: BucketIdInput) -> str:
    """Delete an empty Backblaze B2 bucket by ID."""
    try:
        auth = await _authorize()
        data = await _b2("POST", "b2_delete_bucket", {
            "accountId": auth["accountId"],
            "bucketId": params.bucket_id,
        })
        return f"🗑️ Bucket `{data.get('bucketName', params.bucket_id)}` deleted."
    except Exception as e:
        return f"Error deleting bucket: {e}"

@mcp.tool(name="b2_delete_file",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempotentHint": True},
          async def b2_delete_file(params: DeleteFileInput) -> str:
    """Delete a specific file version from B2. Requires both file ID and file name"""
    try:
        data = await _b2("POST", "b2_delete_file_version", {
            "fileId": params.file_id,
            "fileName": params.file_name,
        })
        return f"🗑️ Deleted: `{data.get('fileName')}` (id=`{data.get('fileId')}`)"
    except Exception as e:
        return f"Error deleting file: {e}"

@mcp.tool(name="b2_hide_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempotentHint": True},
          async def b2_hide_file(params: HideFileInput) -> str:
    """Hide a file in B2 (creates a hide marker – does not delete the file version)"""
    try:
        data = await _b2("POST", "b2_hide_file", {
            "bucketId": params.bucket_id,

```

```

        "fileName": params.file_name,
    })
    return f"🔒 File hidden: `{data.get('fileName')}` (hide marker id=`{data.get('id')}`)"
except Exception as e:
    return f"Error hiding file: {e}"

@mcp.tool(name="b2_copy_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempotentHint": True},
          async def b2_copy_file(params: CopyFileInput) -> str:
    """Copy a B2 file to another bucket or location within B2 (server-side copy)."""
    try:
        data = await _b2("POST", "b2_copy_file", {
            "sourceFileId": params.source_file_id,
            "destinationBucketId": params.dest_bucket_id,
            "fileName": params.dest_file_name,
        })
        return f"📄 Copied to: `{data.get('fileName')}` (id=`{data.get('fileId')}`)"
    except Exception as e:
        return f"Error copying file: {e}"

@mcp.tool(name="b2_upload_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempotentHint": True},
          async def b2_upload_file(params: UploadInput) -> str:
    """Upload a local file to a Backblaze B2 bucket."""
    try:
        if not os.path.exists(params.local_path):
            return f"Error: Local file not found: {params.local_path}"
        dest_key = params.dest_key or os.path.basename(params.local_path)
        with open(params.local_path, "rb") as f:
            content = f.read()
        sha1 = hashlib.sha1(content).hexdigest()
        # Get upload URL
        url_data = await _b2("POST", "b2_get_upload_url", {"bucketId": params.bucket_id})
        upload_url = url_data["uploadUrl"]
        upload_token = url_data["authorizationToken"]
        async with httpx.AsyncClient(timeout=300.0) as client:
            r = await client.post(
                upload_url,
                headers={
                    "Authorization": upload_token,
                    "X-Bz-File-Name": dest_key,
                    "Content-Type": "b2/x-auto",
                    "Content-Length": str(len(content)),
                    "X-Bz-Content-Sha1": sha1,
                },
            )
    except Exception as e:
        return f"Error uploading file: {e}"

```

```

        content=content,
    )
    r.raise_for_status()
    data = r.json()
    return (f"✅ Uploaded **{dest_key}** ({_fmt_size(len(content))}) "
           f"to bucket `{params.bucket_id}`\n"
           f"- File ID: `{data.get('fileId')}`\n"
           f"- SHA1: `{sha1}`")
except Exception as e:
    return f"Error uploading file: {e}"

@mcp.tool(name="b2_download_file",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def b2_download_file(params: DownloadInput) -> str:
    """Download a B2 file by file ID to a local path."""
    try:
        auth = await _authorize()
        dl_url = auth["apiInfo"]["storageApi"]["downloadUrl"]
        token = auth["authorizationToken"]
        url = f"{dl_url}/b2api/v3/b2_download_file_by_id?fileId={params.file_id}"
        async with httpx.AsyncClient(timeout=300.0, follow_redirects=True) as cli
            r = await client.get(url, headers={"Authorization": token})
            r.raise_for_status()
            os.makedirs(os.path.dirname(params.local_path) or ".", exist_ok=True)
            with open(params.local_path, "wb") as f:
                f.write(r.content)
            return f"✅ Downloaded to {params.local_path} ({_fmt_size(len(r.content))}"
    except Exception as e:
        return f"Error downloading file: {e}"

if __name__ == "__main__":
    mcp.run()

```